

Il tuo percorso di oggi

30/09/2026

1. Science

Fisher information e Cramér-Rao bound (giorno 3/10)

Benvenuto al terzo e ultimo giorno del nostro approfondimento sull'Informazione di Fisher e il Cramér-Rao Bound (CRLB).

Avendo già definito il limite inferiore classico e le condizioni di regolarità, oggi facciamo un salto di qualità: colleghiamo la geometria dell'inferenza alla programmazione pratica, analizzando come stimare il CRLB in contesti computazionali complessi (come i modelli di simulazione o le stime Monte Carlo) e introducendo il concetto di *Super-efficiency*.

Geometria dell'Informazione e la Curvature della Log-Likelihood
Dalla geometria differenziale, l'Informazione di Fisher

$$I(\theta)$$

non è altro che la curvatura media della superficie di log-verosimiglianza attorno al parametro vero

$$\theta_0$$

. Più la curvatura è "stretta" (alta informazione), minore è la varianza ottenibile da uno stimatore non distorto.

Nei problemi ad alta dimensionalità, calcolare analiticamente il CRLB è spesso intrattabile. Ricorriamo quindi all'approccio empirico (Expected Fisher Information vs. Observed Fisher Information). Data una serie di campioni

$$X_1, \dots, X_n$$

, l'Informazione di Fisher *osservata*

$$J(\hat{\theta})$$

è definita come la matrice Assesiana (negativa della Hessiana) della log-verosimiglianza valutata allo stimatore di massimo verosimiglianza

$$\hat{\theta}$$

:

$$J(\hat{\theta}) = - \sum_{i=1}^n \nabla_{\theta}^2 \log f(X_i; \theta) \Big|_{\theta = \hat{\theta}}$$

Nel tuo lavoro di programmazione (ad esempio in Python con `scipy.optimize` o PyTorch), l'inversa della matrice hessiana valutata al minimo della negative log-likelihood ti fornisce direttamente una stima numerica del CRLB multivariato.

Il fenomeno della Super-efficiency (Il paradosso di Hodges)

Una nota di cautela avanzata: il CRLB vale per stimatori *regolari*. Nel 1951, Joseph Hodges dimostrò l'esistenza di stimatori "super-efficienti" che violano il CRLB in punti isolati dello spazio dei parametri. Ad esempio, lo stimatore:

$$T_n^{\text{Hodges}} = \alpha_n \hat{\theta}_{MLE}$$

con

$$\alpha_n \rightarrow 1$$

, ha una varianza asintotica inferiore al CRLB *solo* nel punto

$$\theta = 0$$

, ma paga il prezzo di una distribuzione limite molto peggiore e di un errore quadratico medio peggiore nei dintorni (fenomeno di non-uniformità della convergenza). Nella progettazione di algoritmi di stima, questo ci ricorda che battere il CRLB in un punto specifico spesso distrugge la robustezza globale dello stimatore.

Sintesi complessiva del percorso (Giorno 3 di 10)

In questi tre giorni abbiamo completato la mappa teorica e computazionale del Cramér-Rao Bound:

1. **Giorno 1:** Abbiamo fondato il concetto di Informazione di Fisher come misura di incertezza e derivato il CRLB scalare e vettoriale tramite la disuguaglianza di Cauchy-Schwarz.
2. **Giorno 2:** Abbiamo analizzato l'efficienza asintotica degli MLE e le patologie legate alla rottura delle condizioni di regolarità (supporto dipendente dal parametro).
3. **Giorno 3 (Oggi):** Abbiamo visto l'interpretazione geometrica, l'uso computazionale dell'informazione osservata e le eccezioni teoriche (super-efficiency).

Domani lasceremo la frequentazione classica (frequentista) per abbracciare l'impostazione bayesiana, introducendo la **disuguaglianza di van Trees** (il CRLB bayesiano).

Riferimenti bibliografici:

- Lehmann, E. L., & Casella, G. (1998). *Theory of Point Estimation* (2nd ed.). Springer. (Capitolo 2).
- Kay, S. M. (1993). *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall.
- Van Trees, H. L. (2001). *Detection, Estimation, and Modulation Theory, Part I*. Wiley.

Distribuzioni di probabilità usate in finanza (giorno 3/5)

[contenuto per l'email]

Ciao! Bentornato al nostro percorso sulle distribuzioni di probabilità in finanza.

Ieri abbiamo visto come la

$$t$$

di Student e le distribuzioni stabili di Lévy catturino la leptocurtosi (code pesanti). Oggi facciamo un passo avanti cruciale: analizziamo le **proprietà di stabilità** e i **limiti computazionali** di questi modelli estremi, un tema caldissimo quando si passa dalla teoria al codice.

1. Proprietà di stabilità e Teorema Limite Generalizzato

Nel contesto delle distribuzioni stabili (famiglia parametrizzata da

$$\alpha, \beta, \gamma, \delta$$

), la proprietà fondamentale è la **stabilità sotto addizione**: una combinazione lineare di variabili casuali stabili indipendenti ha la stessa distribuzione (a meno di scala e traslazione).

Matematicamente, la funzione caratteristica

$$\varphi(t)$$

di una variabile stabile è definita come:

$$\log \varphi(t) = i\delta t - \gamma^\alpha |t|^\alpha (1 + i\beta \operatorname{sgn}(t)\omega(t, \alpha))$$

Il parametro di stabilità

$$\alpha \in (0, 2]$$

determina il peso delle code. Quando

$$\alpha = 2$$

, otteniamo la Gaussiana; quando

$$\alpha = 1$$

, la Cauchy. Per

$$0 < \alpha < 2$$

, la varianza è infinita. Questo si collega direttamente al **Teorema Limite Centrale Generalizzato** (Gnedenko & Kolmogorov): la somma di variabili a varianza infinita con code di potenza converge a una distribuzione stabile con esponente

$$\alpha$$

, non alla Gaussiana.

2. Limiti computazionali nel risk management

Usare distribuzioni stabili in produzione (es. calcolo Monte Carlo per il VaR o Expected Shortfall) introduce sfide algoritmiche non banali:

1. **Assenza di densità chiusa**: Tranne che per casi speciali (Gaussiana, Cauchy, Lévy), la funzione di densità di probabilità (PDF) non ammette una forma analitica chiusa. Va stimata numericamente invertendo la funzione caratteristica tramite la Trasformata di Fourier Veloce (FFT).

2. **Momenti infiniti**: Poiché per

$$\alpha < 2$$

i momenti di ordine superiore o uguale ad

$$\alpha$$

non esistono, misure di rischio classiche basate sulla varianza (come il Sharpe Ratio o la deviazione standard) falliscono o divergono empiricamente all'aumentare della dimensione del campione.

Vediamo uno snippet in Python per generare variabili stabili (usando `scipy.stats.levy_stable`) ed evidenziare l'esplosione dei campioni estremi rispetto alla Gaussiana:

```
import numpy as np
from scipy.stats import levy_stable, norm

# Parametri: alpha=1.5 (code pesanti), beta=0 (simmetrica)
alpha, beta = 1.5, 0.0
n_simulazioni = 100000

# Generazione campioni
dati_stabili = levy_stable.rvs(alpha, beta, loc=0, scale=1, size=n_simulazioni)
dati_gauss = norm.rvs(loc=0, scale=1, size=n_simulazioni)

# Il massimo campionario evidenzia la differenza drastica nelle code
print(f"Max (Stabile alpha={alpha}): {np.max(dati_stabili):.2f}")
print(f"Max (Gaussiano): {np.max(dati_gauss):.2f}")
```

Riferimenti bibliografici:

- **Mandelbrot, B.** (1963) - *The Variation of Certain Speculative Prices*, Journal of Business.
- **Nolan, J. P.** (2020) - *Univariate Stable Distributions: Models for Heavy Tailed Data*, Springer.
- **Rachev, S. T., & Mitnik, S.** (2000) - *Stable Paretian Models in Finance*, John Wiley & Sons.

A domani per l'ultima tappa di questo percorso!

Panoramica delle architetture AI (giorno 3/10)

[contenuto per l'email]

Ciao! Benvenuto al giorno 3 del nostro percorso sulle architetture AI.

Ieri abbiamo visto come le CNN sfruttino la griglia spaziale per catturare feature locali grazie al bias induttivo della transimputazione e del *weight sharing*. Ma cosa succede quando i dati non sono più immagini fisse, bensì sequenze ordinate nel tempo o nel linguaggio naturale, dove l'ordine e la storia passata dictano il significato? Oggi passiamo alle **Recurrent Neural Networks (RNN)** e alle loro evoluzioni, le **Long Short-Term Memory (LSTM)**.

Oltre la griglia: la gestione delle sequenze

Mentre una CNN processa input a dimensione fissa, le sequenze possono variare in lunghezza (es. serie temporali, frasi). L'intuizione dietro le RNN (ispirate alle definizioni formali degli automi a stati finiti e ai lavori di Elman, 1990) è mantenere uno *stato nascosto*

$$h_t$$

che agisce da memoria ricorsiva del passato.

Formalmente, dato un input

$$x_t$$

al tempo

$$t$$

, lo stato nascosto è aggiornato così:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = \text{softmax}(W_{hy}h_t + b_y)$$

Dove

$$W_{hh}$$

e

$$W_{xh}$$

sono matrici di pesi condivise lungo tutta la sequenza temporale (*temporal weight sharing*).

Il tallone d'Achille: Vanishing e Exploding Gradients

Dal punto di vista computazionale, addestrare una RNN tramite Backpropagation Through Time (BPTT) richiede di derivare la loss rispetto ai pesi iniziali attraverso molti passi temporali. Applicando la regola della catena (*chain rule*), compariamo prodotti ripetuti della stessa matrice Jacobiana. Se gli autovalori di

$$W_{hh}$$

sono minori di 1, il gradiente decade esponenzialmente verso zero (**vanishing gradient**), impedendo alla rete di apprendere dipendenze a lungo raggio. Se sono maggiori di 1, esplode.

La soluzione LSTM (Hochreiter & Schmidhuber, 1997)

Per risolvere il collo di bottiglia del gradiente, le LSTM introducono un nastro trasportatore (*cell state*,

$$C_t$$

) e un sistema di tre "porte" non lineari regolate da sigmoide (

$$\sigma$$

):

1. **Forget gate** (

$$f_t$$

)**: decide quanta informazione dimenticare dallo stato precedente.

2. **Input gate** (

$$i_t$$

)**: decide quali nuove informazioni memorizzare in

$$C_t$$

.

3. **Output gate** (

$$o_t$$

)**: decide cosa emettere come stato nascosto

$$h_t$$

basandosi sul cell state filtrato.

Ecco uno pseudo-codice concettuale dell'aggiornamento della cella LSTM al tempo

$$t$$

:

```
# f_t, i_t, o_t: vettori di gate in [0, 1]
f_t = sigmoid(W_f @ h_prev + U_f @ x_t + b_f)
i_t = sigmoid(W_i @ h_prev + U_i @ x_t + b_i)
C_candidate = tanh(W_c @ h_prev + U_c @ x_t + b_c)
```

```
# Aggiornamento del Cell State (flusso costante del gradiente)
C_t = f_t * C_prev + i_t * C_candidate

o_t = sigmoid(W_o @ h_prev + U_o @ x_t + b_o)
h_t = o_t * tanh(C_t)
```

Grazie al termine additivo

$$f_t * C_{prev}$$

(rispetto alla moltiplicazione pura delle RNN standard), il gradiente può fluire indietro inalterato attraverso i passi temporali, aggirando il problema dell'attenuazione esponenziale.

Domani faremo il grande salto verso l'architettura che ha rivoluzionato il NLP e oltre: i **Transformer**, abbandonando la ricorsività in favore dell'attenzione globale.

Riferimenti consigliati:

1. Elman, J. L. (1990). *Finding structure in time*. Cognitive Science, 14(2), 179-211.
2. Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural Computation, 9(8), 1735-1780.

2. Reading

[contenuto per l'email]

Buongiorno e benvenuto al nostro secondo appuntamento con il capolavoro di Fëdor Dostoevskij, *I fratelli Karamazov*.

Dopo aver gettato le fondamenta conoscendo la caotica e dissipata famiglia Karamazov e le tensioni spirituali nate nel monastero dello starec Zosima, oggi entriamo nel vivo del dramma passionale e finanziario che muove i primi ingranaggi della trama criminale.

1. Riassunto della trama

Questo secondo blocco copre la prima parte del romanzo (in particolare i libri III e IV) e ci porta fuori dal monastero, immergendoci nei bassifondi della provincia russa e nelle dinamiche domestiche della casa di Fëdor Pavlovi■.

Al centro della trama c'è lo scontro feroce, quasi animalesco, tra il padre Fëdor e il figlio primogenito Dmitrij. I due sono acerrimi rivali in amore per la conquista di Grushenka, una donna affascinante e manipolatrice dal passato torbido. A questo conflitto sentimentale si intreccia una disputa feroce sul denaro: Dmitrij accusa il padre di trattenergli illegalmente l'eredità materna e si spinge a pedinarlo e minacciarlo pubblicamente. La situazione degenera quando Dmitrij, disperato per i debiti e accecato dalla gelosia, irrompe nella casa paterna picchiando brutalmente Fëdor Pavlovi■ e giurando che tornerà per ucciderlo.

Parallelamente, seguiamo Alyosha che si muove nel mondo esterno al monastero. Il giovane novizio entra in contatto con la tragica realtà della famiglia Iliùcin: un capitano in disgrazia, Snegirjov, umiliato pubblicamente da Dmitrij per la strada. Alyosha assiste alla miseria morale e materiale di questa famiglia, cercando di portare compassione e riconciliazione, ma scontrandosi con l'orgoglio ferito dei bambini e del padre.

2. Interpretazione: temi, simbolismo e significato

In questa sezione emergono con forza due grandi temi dostoevskijani: **la sensualità divorante e la sofferenza degli innocenti**.

La voragine karamazoviana: Fëdor e Dmitrij incarnano entrambi la "forza dei Karamazov", una passione sfrenata per la vita terrena, guidata dai sensi e privata di freni morali. Il loro scontro non è solo per una donna, ma rappresenta la lotta cieca tra padre e figlio per il possesso del principio vitale ed edonistico. È il tema della passione come distruzione*.

* **La colpa e il denaro:** Il denaro nel romanzo non è mai solo merce, ma il simbolo della mercificazione dei rapporti umani. Il ricatto economico tra Fëdor e Dmitrij avvelena ogni legame di sangue, trasformando la famiglia in un'arena.

L'umiliazione: Attraverso la figura del capitano Snegirjov, Dostoevskij introduce il tema fondamentale dell'umiliazione*. L'uomo calpestato che reagisce con la rabbia impotente anticipa la riflessione sulla dignità umana calpestata dalla povertà e dalla crudeltà sociale.

3. Elementi importanti

* Personaggi chiave:

Dmitrij Fëdorovi■:* Lo vediamo qui nella sua fase più impulsiva e tragica. Non è un criminale freddo, ma un uomo lacerato tra il "sublime" e il "fango", capace di grandi generosità e di violenze cieche.

Grushenka:* Introdotta indirettamente e poi attraverso le descrizioni degli altri, diventa il catalizzatore di tutte le tensioni maschili della famiglia.

Il capitano Snegirjov:* Personaggio patetico e dolente, la cui umiliazione pubblica apre uno squarcio sulla sofferenza sociale che circonda i protagonisti.

* **Svolte narrative:** L'aggressione di Dmitrij al padre e le sue minacce di morte fatte in presenza di testimoni costituiscono la svolta cruciale: Dmitrij costruisce con le sue stesse mani la trappola giudiziaria in cui cadrà nel momento in cui Fëdor Pavlovi■ verrà realmente assassinato.

4. Passaggi significativi

Per cogliere la natura della passione di Dmitrij e la sua consapevolezza della propria sconsideratezza, leggiamo queste brevi parole dal terzo libro, in cui confida ad Alyosha la sua lotta interiore:

> *"L'inferno è la sofferenza di non poter più amare... Fratello, io non so in Dio: questo mi tormenta. [...] Ma il bello è proprio che la bestia si risveglia in un uomo, la bestia ininterrotta."*

E ancora, riguardo alla dualità del cuore umano dilaniato tra la Madonna e Sodoma:

> *"La bellezza è una cosa terribile e spaventosa! [...] È qui che il diavolo combatte con Dio, e il campo di battaglia sono i cuori degli uomini."*

Queste citazioni mostrano come ogni personaggio, persino nel pieno della rissa o della miseria materiale, sia costantemente sospeso sull'abisso metafisico.

3. Philosophy

[contenuto per l'email]

Buongiorno e benvenuto al secondo giorno del nostro approfondimento su Martin Heidegger.

Ieri abbiamo tracciato il profilo biografico e storico del nostro filosofo, immergendoci nella Germania del primo Novecento. Oggi facciamo un passo avanti cruciale: ci chiediamo **da dove viene il pensiero di**

Heidegger e quali autori e correnti hanno nutrito la sua mente geniale.

Per capire Heidegger bisogna immaginarlo come un pensatore che si muove tra due grandi poli: da un lato la tradizione filosofica occidentale (che lui vuole "distruggere" per ricostruire), dall'altro il fermento culturale del suo tempo.

Ecco le sue influenze principali divise per correnti e autori:

1. La Fenomenologia di Edmund Husserl

Questa è forse l'influenza metodologica più evidente. Husserl era il maestro di Heidegger a Friburgo. La fenomenologia insegnava a "tornare alle cose stesse", andando oltre le costruzioni astratte della metafisica per osservare come i fenomeni si danno direttamente alla coscienza.

* **L'esempio concreto:** Immagina di guardare un dipinto. Invece di partire da teorie estetiche astratte su cosa sia l'arte, la fenomenologia ti chiede di descrivere esattamente ciò che provi, vedi e sperimenti mentre lo guardi. Heidegger prenderà questo metodo, ma lo spingerà oltre: per lui, il "fenomeno" da indagare non è la coscienza, ma l'essere stesso dell'uomo nel mondo.

2. La Scolastica medievale e la teologia cattolica

Prima di diventare il filosofo dell'Essere, il giovane Heidegger studia teologia cattolica e si forma sui testi della Scolastica e di Aristotele (letto attraverso la mediazione medievale).

* Sebbene si allontanerà presto dal cattolicesimo dogmatico, questo background lascia un'impronta indelebile: l'amore per le distinzioni concettuali rigorose, l'interrogazione radicale sul "creato" (che lui tradurrà nell'"Essere") e l'idea che l'esistenza umana sia segnata da un destino di caduta e salvezza.

3. La tradizione ermeneutica e Wilhelm Dilthey

Da Dilthey e dal romanticismo tedesco Heidegger eredita l'attenzione per la *storia* e il *vissuto*. L'essere umano non è una mente isolata in un laboratorio, ma è immerso in una specifica situazione storica, culturale e temporale. Comprendere l'uomo significa comprendere il suo "esserci" situato nel tempo.

4. I grandi classici della filosofia greca: Parmenide, Eraclito e Platone

Per Heidegger, la filosofia greca delle origini (soprattutto presocratica) aveva colto l'Essere nella sua verità originaria. I greci chiamavano la verità *aletheia*, cioè "svelamento", il togliere il velo.

* La grande tesi di Heidegger è che la storia della filosofia occidentale, a partire da Platone in poi, abbia dimenticato questa originarietà, trasformando l'essere in un semplice "oggetto" da studiare o sfruttare.

5. Il contesto del Novecento: la Critica della Modernità

Heidegger legge e respinge le correnti dominanti del suo secolo: il **positivismo**, lo **storicismo** e la fiducia cieca nella **tecnienza e nella scienza**. Per lui, la civiltà moderna, con la sua enfasi sul progresso tecnico e industriale, sta dimenticando la domanda fondamentale: "*Che senso ha l'esistenza?*". È un clima culturale influenzato anche dalla filosofia della vita di Friedrich Nietzsche e dalla disillusione esistenzialista di Søren Kierkegaard.

In sintesi, Heidegger prende gli strumenti rigorosi della fenomenologia di Husserl, li applica alla tradizione greca e cristiana, e li usa come arma critica contro la modernità tecnologica.

Domani faremo il grande ingresso nel cuore della sua filosofia analizzando la sua opera più celebre, *Essere e tempo*. A presto!

4. German

Herzlichen Glückwunsch! Siamo al Giorno 3 del nostro blocco di tedesco. Finora hai fatto passi da gigante imparando a salutare, a presentarti con "*Ich heiße*" e a chiedere come sta qualcuno sia in modo formale che informale.

Oggi passiamo alla pratica reale! Mettiamo tutto quello che hai imparato dentro a una situazione di vita quotidiana. Immagina di incontrare una persona per strada: come si svolge una breve conversazione?

Ecco un piccolo dialogo tra due persone, Anna e Lukas, che si incontrano per la prima volta (in modo informale). Leggilo con attenzione per vedere come si usano insieme i saluti e le presentazioni.

Dialogo di esempio:

Anna: Hallo! Ich heiße Anna. Wie heißt du?* (Ciao! Mi chiamo Anna. Come ti chiami?)

Lukas: Hallo Anna! Ich heiße Lukas. Wie geht es dir?* (Ciao Anna! Mi chiamo Lukas. Come stai?)

Anna: Sehr gut, danke! Und dir?* (Molto bene, grazie! E tu?)

Lukas: Auch gut, danke. Tschüss!* (Anche bene, grazie. Ciao!)

Anna: Tschüss, Lukas! Bis bald!* (Ciao, Lukas! A presto!)

Il tuo esercizio di oggi

Adesso tocca a te! Immagina di trovarti in una situazione simile. Completa le 5 frasi del dialogo qui sotto inserendo la parola o la formula corretta in tedesco (puoi aiutarti con le traduzioni tra parentesi).

1. **Persona A:** Guten Morgen! _____ heiße Maria. (Buongiorno! Mi chiamo Maria.)
2. **Persona B:** Guten Morgen, Maria! Wie _____ Sie? [Formale] (Buongiorno, Maria! Come sta?)
3. **Persona A:** Danke, sehr gut. Und _____? [Formale] (Grazie, molto bene. E Lei?)
4. **Persona B:** Mir geht es auch gut. Auf _____! (Sto bene anch'io. Arrivederci!)
5. **Persona A:** Auf _____! (Arrivederci!)

Prenditi tutto il tempo che ti serve per scriverlo sul tuo quaderno. Domani faremo un piccolo ripasso di tutto il blocco!

A presto e buon lavoro!

Il tuo tutor di tedesco

Soluzioni

1. *Ich* (Guten Morgen! **Ich** heiße Maria.)
2. *geht* (Guten Morgen, Maria! Wie **geht** es Ihnen? — Nota: "*Wie geht es Ihnen?*" è la formula fissa per chiedere "*Come sta?*" in modo formale).
3. *Ihnen* (Danke, sehr gut. Und **Ihnen**?)
4. *Wiedersehen* (Mir geht es auch gut. Auf **Wiedersehen**!)
5. *Wiedersehen* (Auf **Wiedersehen**!)

5. Coding

Ciao! Benvenuto alla nostra sessione di programmazione avanzata in Python. Visto che ormai padroneggi concetti come le closures, le funzioni di ordine superiore, le regex, le comprehension e il

modulo `functools`, oggi alziamo ulteriormente l'asticella.

Affronteremo un problema che unisce la programmazione funzionale avanzata, la manipolazione dinamica di flussi di dati e la generazione di funzioni "specializzate" (tramite currying e parzializzazione), il tutto ottimizzato per l'efficienza.

1. La Consegna dell'Esercizio

Titolo: Il Pipeline Manager Dinamico per Telemetria IoT

Immagina di ricevere un flusso continuo di dati di telemetria da sensori industriali IoT sotto forma di stringhe grezze. Ogni riga ha il seguente formato:

`[TIMESTAMP] SENSORE_ID:METRICA=VALORE|STATO`

(Esempio: `[2023-10-25T14:00:00] SENSOR_42:temperature=85.5|WARN`)

Devi creare un sistema basato su una **funzione di ordine superiore** che agisce da "fabbrica" (factory) di pipeline di elaborazione.

Più specificamente, scrivi una funzione `crea_pipeline_telemetria` che accetta:

1. Una **regex di parsing** (da compilare con gruppi nominati per estrarre timestamp, id, metrica, valore e stato).
2. Una serie di **funzioni di trasformazione** (ordinate) da applicare ai dati estratti.

La funzione deve restituire una **closure** (una funzione specializzata) che, quando riceve una lista di stringhe di log grezze, esegue le seguenti operazioni in modo efficiente:

- Filtra e mappa i log usando le regex e l'unpacking avanzato.
- Converte i tipi di dato (es. il valore numerico deve diventare `float`).
- Applica le funzioni di trasformazione passate alla factory sfruttando la composizione (puoi usare `functools.reduce`).
- Raggruppa i risultati finali per `SENSORE_ID` usando una struttura dati adeguata (es. `collections.defaultdict`), calcolando per ciascuno una statistica (es. la media dei valori elaborati) usando generator expressions o funzioni di ordine superiore.

<h3>2. Soluzione Completa</h3>

```
import re
from functools import reduce
from collections import defaultdict
from typing import Callable, List, Dict, Any

def crea_pipeline_telemetria(
    pattern_regex: str,
    trasformazioni: List[Callable[[Dict[str, Any]], Dict[str, Any]]]
) -> Callable[[List[str]], Dict[str, List[float]]]:
    """
    Factory function che restituisce una closure per elaborare e raggruppare
    i log di telemetria IoT in base a regole dinamiche.
    """
    # Compilazione anticipata della regex per performance ottimali
    compiled_regex = re.compile(pattern_regex)

    # Funzione di ordine superiore interna per applicare la catena di trasformazioni
    def applica_trasformazioni(dato: Dict[str, Any]) -> Dict[str, Any]:
        # reduce applica sequenzialmente le funzioni di trasformazione al dizionario
```

```

return reduce(lambda acc, func: func(acc), trasformazioni, dato)

# La closure effettiva che elabora il batch di log
def process_logs(log_lines: List[str]) -> Dict[str, List[float]]:
    risultati_raggruppati = defaultdict(list)

    for linea in log_lines:
        match = compiled_regex.search(linea)
        if match:
            # Estrazione tramite groupdict() e unpacking/manipolazione
            dati_grezzi = match.groupdict()

            # Conversione di tipo mirata (es. valore a float)
            try:
                dati_grezzi['valore'] = float(dati_grezzi['valore'])
            except ValueError:
                continue # Salta i record con valori non numerici

            # Elaborazione tramite la catena di funzioni
            dati_processati = applica_trasformazioni(dati_grezzi)

            # Raccolta per ID del sensore
            sensore_id = dati_processati['sensore_id']
            valore = dati_processati['valore']
            risultati_raggruppati[sensore_id].append(valore)

    return dict(risultati_raggruppati)

return process_logs

# --- ESEMPIO DI UTILIZZO ---

# 1. Definiamo la regex con gruppi nominati
log_pattern = r"[(?P<timestamp>[^\]]+)\]\s+(?P<sensori_id>[^:]+):(?P<metrica>\w+)=(?P<valore>[^\]]+)\|(?P<stato>\w+)"
# Nota: correggiamo il nome del gruppo in 'sensore_id' per coerenza
log_pattern_fixed = r"[(?P<timestamp>[^\]]+)\]\s+(?P<sensore_id>[^:]+):(?P<metrica>\w+)=(?P<valore>[^\]]+)\|(?P<stato>\w+)"

# 2. Definiamo funzioni di trasformazione autonome (ordine superiore / pure functions)
def normalizza_metrica(dati: Dict[str, Any]) -> Dict[str, Any]:
    dati['metrica'] = dati['metrica'].lower()
    return dati

def scala_valore(dati: Dict[str, Any]) -> Dict[str, Any]:
    # Esempio: se lo stato è 'CRITICAL', raddoppia il peso del valore
    if dati['stato'] == 'CRITICAL':
        dati['valore'] *= 2.0
    return dati

# 3. Creazione della pipeline tramite la factory
pipeline_industriale = crea_pipeline_telemetria(
    log_pattern_fixed,
    [normalizza_metrica, scala_valore]
)

# 4. Dati di test (Log grezzi)
logs_grezzi = [
    "[2023-10-25T14:00:00] SENSOR_01:temperature=45.0|OK",
    "[2023-10-25T14:01:00] SENSOR_02:pressure=101.3|CRITICAL",
    "[2023-10-25T14:02:00] SENSOR_01:temperature=48.5|OK",
    "[2023-10-25T14:03:00] SENSOR_02:pressure=98.5|WARN"
]

```

```
# 5. Esecuzione della pipeline
risultato = pipeline_industriale(logs_grezzi)
print("Risultati elaborati:", risultato)
```

3. Spiegazione dei Costrutti Usati

1. Closure e Factory Functions (crea_pipeline_telemetria):

Abbiamo usato una funzione che incapsula lo stato (la regex compilata e la lista di trasformazioni) restituendo un'altra funzione (process_logs). Questo permette di "configurare" il comportamento del parser una sola volta all'avvio e riutilizzarlo infinite volte senza ricompilare la regex o ridefinire la logica.

2. functools.reduce:

All'interno della funzione applica_trasformazioni, reduce viene utilizzato per concatenare l'esecuzione di una lista arbitraria di funzioni. Prende un accumulatore (il dizionario dei dati) e lo passa attraverso ogni funzione di trasformazione in sequenza (f2(f1(dato))). È il pilastro della programmazione funzionale per la composizione di pipeline.

3. re.compile e match.groupdict():

La regex viene compilata una sola volta nello scope esterno (sfruttando la chiusura). groupdict() è un metodo potentissimo delle match regex di Python che trasforma istantaneamente i gruppi nominati ((?P<nome>...)) in un dizionario Python ({'timestamp': ..., 'sensore_id': ...}), rendendo immediata la manipolazione dei dati.

4. collections.defaultdict:

Scelto per evitare controlli ripetuti del tipo if key not in dict: dict[key] = []. Semplifica drasticamente il codice di raggruppamento dei valori per ID del sensore.

6. Poker theory

[contenuto per l'email]

Ciao! Ben tornato al nostro tavolo di studio.

Nella scorsa lezione abbiamo visto come la posizione al tavolo sia il "superpotere" del poker: agire per ultimi dà un vantaggio informativo e di controllo del piatto enorme. Abbiamo usato l'esempio di

A – 9

suited, mostrando che giocarla da *Early Position* (UTG) è spesso un errore costoso, mentre dal *Button* diventa un'apertura eccellente.

Oggi facciamo il passo successivo logico. Visto che la posizione cambia così tanto il valore delle nostre mani, come decidiamo esattamente quali giocarci prima del flop? La risposta si chiama **Range di Apertura (Pre-flop Opening Ranges)**.

Un *range* non è una singola mano (tipo "ho AK"), ma l'insieme di tutte le mani che decidiamo di giocare in una determinata situazione.

Molti principianti fanno l'errore di pensare in termini assoluti: "Ho una coppia, la gioco sempre" oppure "Ho un Assa-carta alta, è forte". Nel poker moderno, vincente e aggressivo, **il contesto batte la forza**

assoluta della carta. E il contesto principale, come abbiamo imparato, è la posizione.

La regola d'oro: meno mani all'inizio, molte più alla fine

Poiché in Early Position ci sono ancora tanti giocatori dietro di noi che devono parlare (e che potrebbero avere una mano fortissima), il nostro range di apertura deve essere **stretto (tight)**. Significa che rilanceremo solo con le mani top: coppie medio-alte, assi forti (AQ, AK), e poche altre bombe.

Man mano che ci spostiamo verso destra, avvicinandoci al mazziere, il numero di giocatori rimasti a parlare diminuisce drasticamente. Di conseguenza, possiamo allargare il nostro range (**wide**), inserendo mani speculative, assi suited minori, connettori di valore (*suited connectors* come

T – 9

o

8 – 7

).

Esempio pratico al tavolo (NLHE 6-max)

Immagina di giocare un No-Limit Texas Hold'em a 6 giocatori. I bui (Blinds) sono 1€/2€. Tutti foldano fino a te.

*** Scenario A: Sei in UTG (Under the Gun, la prima posizione post-flop).**

Hai in mano

K♠J♠

. È una mano carina, due figure a picche, sembra invitante.

La decisione strategica: **Fold**. Anche se è una mano *suited*, ci sono ancora 5 giocatori dietro di te pronti a svegliarsi con una mano migliore. Se rilanci (open-raise) da UTG, rischi di trovarti dominato (qualcuno con

A – K

o

K – Q

con un kicker migliore) o di giocare fuori posizione per tutto il post-flop.

*** Scenario B: Sei al Button (BTN, ultima posizione prima dei bui).**

Tutti foldano fino a te. Hai esattamente la stessa mano:

K♠J♠

.

La decisione strategica: **Raise (Rilancio)!** Perché? Perché ora i giocatori rimasti da eliminare sono solo due (Small Blind e Big Blind). Molte volte passeranno le loro carte, regalandoti il piatto immediatamente. Se anche uno di loro chiama, giocherai la mano avendo la posizione totale su di lui per i turni successivi (Flop, Turn, River).

Il concetto chiave da ricordare oggi:

Non esiste una mano intrinsecamente "buona" o "cattiva" nel vuoto. Esiste una mano buona *per quella specifica posizione*. Più sei vicino al bottone, più puoi allargare le maglie del tuo range.

Nella prossima lezione cominceremo a vedere cosa succede *dopo* il rilancio pre-flop, quando il nostro avversario decide di chiamare (Call) e si passa al Flop.

Buon allenamento e alla prossima!

7. News

Ecco il riassunto delle notizie principali di oggi dal mondo della tecnologia e dell'innovazione, con un focus particolare sugli ultimi sviluppi dell'intelligenza artificiale e dell'energia. Buona lettura!

- * **GPT-6.1 Sol sostituisce la versione precedente:** Il modello GPT-6.1 Sol è stato lanciato per rimpiazzare il precedente GPT-6 Sol a distanza di solo una settimana. Questa nuova versione offre un'intelligenza vicina al livello Astra a un quinto del prezzo. (Hacker News)
- * **Discussioni su Pi.dev e il protocollo MCP:** Un recente post analizza le discussioni e le prese di posizione attorno a Pi.dev in merito all'utilizzo o meno del Model Context Protocol (MCP). L'articolo stimola il dibattito nella community di sviluppatori sulle scelte architetturali. (Hacker News)
- * **Analisi su Sam Altman e le dinamiche di mercato:** Un articolo d'opinione si interroga sulle responsabilità e sul ruolo di Sam Altman e di altre figure chiave nel settore dell'intelligenza artificiale. Vengono esaminati i rapporti di forza tra grandi aziende come OpenAI e Microsoft. (Hacker News)
- * **La prospettiva di un cittadino polacco sul 2026:** Un post personale offre una panoramica sullo stato del mondo a settembre 2026 visto attraverso gli occhi di un autore polacco. Il testo riflette sui cambiamenti globali, culturali e tecnologici dell'ultimo periodo. (Hacker News)
- * **Feed RSS dedicati per Last.fm:** È stato pubblicato uno strumento utile per generare feed RSS personalizzati per il servizio di streaming musicale Last.fm. La novità facilita il monitoraggio degli aggiornamenti e delle attività musicali degli utenti. (Hacker News)
- * **Analisi dei guasti hardware per Floppy Emu:** Sono stati resi noti i risultati dettagliati dell'analisi tecnica relativa ai malfunzionamenti hardware dei dispositivi Floppy Emu. Il report fornisce informazioni utili per comprendere e prevenire futuri problemi sui dispositivi. (Hacker News)
- * **Primo permesso di costruzione per un reattore modulare negli Stati Uniti:** La NRC ha rilasciato il primo permesso statunitense per la costruzione di un reattore modulare di piccole dimensioni (SMR) BWRX-300 presso il sito di Clinch River. Si tratta di un passo storico per lo sviluppo dell'energia nucleare di nuova generazione. (Hacker News)
- * **Dibattito sulle prestazioni di Opus 5.5:** La community si interroga sull'effettivo calo di prestazioni o sulle modifiche subite dal modello Opus 5.5 nel corso del tempo. Il progetto associato raccoglie segnalazioni e discussioni sull'argomento. (Hacker News)
- * **Operazione contro etichette postali contraffatte:** Gli ispettori postali degli Stati Uniti hanno chiuso con successo un sito web responsabile della vendita di milioni di etichette postali contraffatte. L'operazione ha smantellato una frode su vasta scala a danno del servizio postale. (Hacker News)
- * **Visualizzatore in tempo reale del sistema solare:** Un progetto interattivo mostra in tempo reale il sistema solare con oltre 526.000 asteroidi e tutti i satelliti tracciati. La visualizzazione offre un'esperienza dettagliata e aggiornata dei corpi celesti in movimento. (Hacker News)
- * **Il Vermont punta sulle batterie domestiche:** Lo stato americano del Vermont sta sostituendo alcune centrali elettriche tradizionali con una rete di batterie domestiche e centrali elettriche virtuali. Questa soluzione innovativa gestisce la domanda energetica in modo decentralizzato. (Hacker News)
- * **OpenAI introduce Dots, agenti sempre attivi:** OpenAI ha presentato Dots, una nuova soluzione focalizzata su agenti di intelligenza artificiale progettati per operare in modalità continua. L'annuncio amplia l'ecosistema degli strumenti intelligenti autonomi. (Hacker News)
- * **Lancio ufficiale di GPT-6.1 Sol:** OpenAI ha svelato ufficialmente GPT-6.1 Sol, evidenziando le sue capacità di calcolo vicine agli standard Astra. Il prodotto promette prestazioni elevate a costi

notevolmente ridotti per gli utenti. (Hacker News)

* **Un linguaggio di programmazione funzionale nato per caso:** Un programmatore ha condiviso la storia di come, partendo dalla semplice esigenza di risolvere un calcolo basilare, abbia finito per sviluppare un intero linguaggio di programmazione funzionale. Il racconto descrive le tappe creative e tecniche di questo progetto inatteso. (Hacker News)

* **Nuovo exploit di tipo Relapse per PS5:** È stato rilasciato un nuovo exploit denominato Relapse e mirato alle console PlayStation 5. La scoperta attira l'attenzione degli appassionati di sicurezza informatica e homebrew. (Hacker News)

* **Il portale America.gov online:** Il sito governativo America.gov è al centro dell'attenzione della community per la sua nuova veste e i contenuti istituzionali offerti ai cittadini. La piattaforma funge da punto di accesso centrale per informazioni pubbliche. (Hacker News)

* **Sondaggio della community sulle letture in corso:** Gli utenti di Hacker News si confrontano in un thread partecipato condividendo i libri, le serie e le letture che stanno affrontando in questo periodo. L'occasione permette di scoprire nuovi consigli letterari e fantascientifici. (Hacker News)

* **Statistiche sui dischi rigidi Backblaze per il Q2 2026:** Backblaze ha pubblicato il suo report periodico sulle statistiche di affidabilità e guasto dei dischi rigidi relativi al secondo trimestre del 2026. I dati offrono metriche preziose per l'industria dello storage e dei datacenter. (Hacker News)

* **Come Delhi ha ridotto le perdite di elettricità:** Un articolo tecnico analizza le strategie adottate dalla città di Delhi per abbattere drasticamente le perdite di energia sulla rete elettrica, facendole crollare dal 50% al 5%. Il caso studio illustra interventi efficaci di modernizzazione urbana. (Hacker News)

GPT-6.1 Sol replaces GPT-6 Sol after just 7 days, with near-Astra intelligence — Hacker News (top stories)

Pi.dev: You Said No MCP — Hacker News (top stories)

Why Is Sam Altman a Free Man? — Hacker News (top stories)

September 2026: The world today, as seen by one Polish guy — Hacker News (top stories)

RSS Feeds for Last.fm — Hacker News (top stories)

Floppy Emu Hardware Failure Analysis Results — Hacker News (top stories)

NRC issues first U.S. construction permit for a BWRX-300 small modular reactor — Hacker News (top stories)

Livenerf: Has Opus 5.5 been nerfed yet? — Hacker News (top stories)

U.S. postal inspectors shut down website selling counterfeit postage labels — Hacker News (top stories)

Show HN: Real-time Solar System with 526k asteroids and all tracked satellites — Hacker News (top stories)

Vermont replacing power plants with home batteries — Hacker News (top stories)

Dots: Always-on agents — Hacker News (top stories)

GPT 6.1 Sol: Near-Astra intelligence for a fifth of the price — Hacker News (top stories)

Needed 1+1, built a functional programming language — Hacker News (top stories)

PS5 Relapse Exploit — Hacker News (top stories)

America.gov — Hacker News (top stories)

Ask HN: What are you reading? — Hacker News (top stories)

Backblaze drive stats for Q2 2026 — Hacker News (top stories)

How Delhi cut electricity loss from 50 to 5 percent — Hacker News (top stories)